

Routing Algorithms for 2-D Network-on-Chip

Implementation and Performance Evaluation using Noxim Simulator



BITS Pilani, K.K. Birla Goa Campus

Advanced Computer Architecture Project

Submitted by:

Yashraj Verma — 2025H1030028G

Mohd. Abdul Junaid — 2025H1030141G

April 2026

Abstract

This report presents the design, implementation, and comprehensive performance evaluation of three routing algorithms for a 2D Mesh Network-on-Chip (NoC): the deterministic XY Routing, the partially adaptive West-First Routing, and a novel enhancement termed West-First with Congestion Threshold (WF+CT), developed as part of this work. The objective of this study is to analyze how different routing strategies impact network performance under varying traffic conditions, with a particular focus on latency, throughput, and saturation behavior.

All simulations were conducted using the Noxim SystemC-based NoC simulator on a 4×4 mesh topology, a commonly used architecture for evaluating on-chip interconnection networks. The evaluation considered two widely studied traffic patterns—Uniform Random traffic, which distributes packets evenly across nodes, and Hotspot traffic, which introduces non-uniform load by directing a significant portion of traffic to specific nodes. Packet Injection Rate (PIR) was varied from 0.01 to 0.20 to observe system behavior across low-load, near-saturation, and post-saturation regions.

The results demonstrate clear trade-offs between simplicity, adaptability, and performance. Under Uniform Random traffic, XY routing achieves the highest post-saturation throughput, reaching up to 4.43 flits per cycle, primarily due to its minimal and deterministic path selection, which reduces routing overhead. However, its lack of adaptability leads to early saturation under non-uniform conditions, particularly in the Hotspot scenario where it saturates at a PIR of 0.02.

In contrast, the West-First routing algorithm introduces partial adaptivity by allowing alternative minimal paths while maintaining deadlock freedom. Building on this, the proposed WF+CT algorithm incorporates congestion awareness by dynamically selecting routes based on a predefined congestion threshold. This enhancement results in improved performance under high-contention scenarios. Notably, at the critical PIR of 0.02 in Hotspot traffic, WF+CT reduces average packet latency to 599 cycles compared to 715 cycles for standard West-First, demonstrating the effectiveness of congestion-aware routing decisions.

Across Uniform Random traffic, all three algorithms exhibit saturation at $\text{PIR} = 0.04$. However, WF+CT shows improved saturation latency (1904 cycles) compared to West-First (2137 cycles), indicating better handling of network congestion even under balanced traffic. These findings highlight that while deterministic routing excels in stable and uniform conditions, adaptive and congestion-aware approaches provide significant advantages in realistic, uneven traffic scenarios.

1. Introduction

Modern multi-core processors integrate tens to hundreds of processing elements on a single chip. As core counts grow, traditional bus-based interconnects fail to scale due to bandwidth limitations, increased latency, and bus contention. Network-on-Chip (NoC) architectures address this by organizing processing elements as nodes in a structured topology — typically a 2D mesh — where dedicated routers forward packets between nodes using defined routing algorithms.

The choice of routing algorithm critically determines NoC performance. Deterministic algorithms such as XY routing are simple and deadlock-free but cannot adapt to congestion. Under hotspot or unbalanced traffic, they may over-utilize certain links while leaving others idle, leading to premature network saturation. Adaptive routing algorithms attempt to distribute load more evenly but introduce complexity and potential deadlock risk.

This project implements and evaluates three routing strategies:

- XY Routing — deterministic, used as the performance baseline
- West-First Routing — a partially adaptive, turn-model-based algorithm that is deadlock-free
- West-First + Congestion Threshold (WF+CT) — our custom extension that filters routing directions based on real-time buffer occupancy

All implementations were done within the Noxim open-source NoC simulator and evaluated on metrics of average packet latency, network throughput, and saturation injection rate.

2. Background and Related Concepts

2.1 2D Mesh Topology

A 2D mesh NoC arranges $N \times N$ routers in a grid. Each router connects to up to 4 neighbors (North, South, East, West) and one local processing element. Packets traverse the mesh hop-by-hop. In this project, a 4x4 mesh (16 nodes) was used, which is a standard evaluation topology in NoC research.

2.2 XY Routing

XY (also called Dimension-Order Routing) routes packets first along the X-axis (East or West) until the destination column is reached, then along the Y-axis (North or South). It is deterministic — every packet between a given source-destination pair always takes the same path. This guarantees deadlock-freedom but cannot react to congestion, causing hotspot links to saturate early.

2.3 West-First Routing

West-First is a partially adaptive turn-model algorithm. Its core constraint: westward movement must always be performed first and deterministically. Once all required westward movement is complete, the router may adaptively choose between North/South and East directions. This constraint prevents cyclic dependencies and thus guarantees deadlock-freedom while allowing limited adaptive behavior.

2.4 Congestion-Aware Routing

Congestion-aware routing extends an adaptive algorithm by incorporating real-time network state — specifically buffer occupancy — into routing decisions. When multiple output directions are valid, the algorithm checks buffer fill levels and avoids heavily congested ports. Our WF+CT implementation adds this layer to West-First routing: when two adaptive directions are available, directions whose buffers are more than 75% full are filtered out, with a safety fallback ensuring deadlock-freedom is preserved.

3. Implementation

3.1 Simulator: Noxim

Noxim is an open-source SystemC-based NoC simulator developed at the University of Catania. It provides a configurable 2D mesh topology, router and buffer implementation, traffic generation (uniform, hotspot, table-based), and automatic collection of performance metrics. All experiments use:

- Mesh size: 4x4 (16 nodes)
- Buffer depth: 4 flits per port
- Packet size: 2-10 flits (default)
- Simulation time: 10,000 cycles (+ 1,000 cycle reset period)
- Selection strategy: RANDOM (for West-First and WF+CT)

3.2 XY Routing (Baseline)

XY routing was used as-is from the Noxim codebase (Routing_XY.cpp). No modifications were required. It served as the performance reference against which all other algorithms were compared.

3.3 West-First Routing

West-First routing was also available in the Noxim codebase (Routing_WEST_FIRST.cpp). The algorithm logic is:

1. If the destination is to the West OR in the same column as the current node, delegate to XY routing (deterministic westward and straight movement).
2. If the destination is to the East and North, return [DIRECTION_NORTH, DIRECTION_EAST] as valid options.
3. If the destination is to the East and South, return [DIRECTION_SOUTH, DIRECTION_EAST] as valid options.

When multiple directions are returned, the selection strategy (RANDOM) picks one. This is the adaptive component.

3.4 West-First + Congestion Threshold (WF+CT)

WF+CT was implemented as a new routing module: Routing_WEST_FIRST_CT.cpp. It extends West-First with the following additional step applied after the adaptive direction list is built:

4. For each candidate direction, query the output buffer occupancy: `router->buffer[dir][0].getCurrentFreeSlots()` and `GetMaxBufferSize()`.
5. If free slots > 25% of max buffer size (i.e., buffer is less than 75% full), the direction is kept as uncongested.
6. The filtered uncongested list is returned. If ALL directions are congested (filtered list is empty), the original full list is returned as a safety fallback — ensuring the algorithm never deadlocks by refusing all directions.

The threshold of 75% was chosen to activate congestion avoidance only when buffers are significantly loaded, avoiding unnecessary filtering under light traffic. This also required registering the new algorithm in the Noxim power model (`power.yaml`) using identical power parameters to West-First.

4. Experimental Setup

4.1 Traffic Patterns

Two traffic patterns were evaluated:

- Uniform Random Traffic (TRAFFIC_RANDOM): Each node sends packets to uniformly randomly chosen destinations. This tests general-purpose performance and saturation behavior.
- Hotspot Traffic: Node 0 is designated as a hotspot receiver with 20% extra traffic probability (-hs 0 0.2). This creates an unbalanced load that stresses congestion management.

4.2 Injection Rate Sweep

Each algorithm was run at 11 packet injection rates (PIR): 0.01, 0.02, 0.04, 0.06, 0.08, 0.10, 0.12, 0.14, 0.16, 0.18, 0.20. This sweep captures pre-saturation, saturation, and post-saturation behavior.

4.3 Metrics Collected

- Global average delay (cycles) — average latency from packet injection to reception
- Network throughput (flits/cycle) — total flits delivered per simulation cycle
- Saturation injection rate — the PIR at which latency exhibits a sharp increase

5. Results and Analysis

5.1 Uniform Traffic — Latency

Table 1 shows average latency across PIR values for all three algorithms under uniform traffic.

Algorithm	PIR 0.01	PIR 0.02	PIR 0.04 (sat.)	PIR 0.08	PIR 0.12	PIR 0.20
XY	11.88	20.14	838.5	3059	3838	4431
WF	11.37	30.09	2137	3717	4189	4600
WF+CT	12.14	22.62	1904	3633	4303	4666

Table 1: Average Latency (cycles) — Uniform Traffic

Key observations:

- All three algorithms operate with nearly identical low latency (~11-12 cycles) at PIR=0.01, confirming correct implementation and fair comparison baseline.
- XY saturates sharply at PIR=0.04 (latency 838 cycles), but earlier than WF and WF+CT in absolute terms — while its post-saturation latency is higher, its throughput is also higher.
- WF saturates at PIR=0.04 with latency 2137 cycles — significantly worse than XY at the same point.
- WF+CT saturates at PIR=0.04 with latency 1904 cycles — an 11% improvement over plain WF at the saturation boundary, demonstrating that congestion filtering delays the worst-case latency rise.

5.2 Uniform Traffic — Throughput

Algorithm	PIR 0.01	PIR 0.02	PIR 0.04	PIR 0.08	PIR 0.12	PIR 0.20
XY	1.28	2.51	4.18	4.10	4.16	4.06
WF	1.23	2.51	3.07	2.83	2.99	3.06
WF+CT	1.27	2.45	2.92	2.87	2.92	2.90

Table 2: Network Throughput (flits/cycle) — Uniform Traffic

- XY achieves significantly higher post-saturation throughput (~4.1 flits/cycle) compared to WF and WF+CT (~2.9-3.1 flits/cycle). This is because XY's deterministic shortest paths are more efficient in routing overhead.
- WF and WF+CT have comparable throughput, with WF+CT being marginally lower in some cases — the congestion filtering occasionally restricts directions that would have been productive, a known tradeoff of conservative adaptive routing.

5.3 Hotspot Traffic — Latency

Algorithm	PIR 0.01	PIR 0.02 (sat.)	PIR 0.04	PIR 0.08	PIR 0.12	PIR 0.20
XY	13.91	689.5	1719	3268	3855	4462
WF	19.58	715.8	1802	3665	4157	4667
WF+CT	19.55	599.6	2080	3634	4111	4609

Table 3: Average Latency (cycles) — Hotspot Traffic

This is the most revealing result set:

- XY saturates earliest under hotspot — at PIR=0.02 (latency 689 cycles vs 13 at PIR=0.01). Hotspot traffic concentrates on fixed deterministic paths, and XY cannot reroute around the congested node 0.
- WF also saturates at PIR=0.02 (latency 715 cycles) — slightly worse than XY at this point because its adaptive behavior under hotspot conditions can lead to suboptimal path selection.
- WF+CT achieves 599 cycles at PIR=0.02 — 13% lower latency than plain WF and 15% lower than XY. This is the clearest evidence that congestion-aware direction filtering provides measurable benefit under hotspot conditions, where buffers near node 0 fill up and the CT logic actively redirects traffic.

5.4 Hotspot Traffic — Throughput

Algorithm	PIR 0.01	PIR 0.02	PIR 0.04	PIR 0.08	PIR 0.12	PIR 0.20
XY	1.23	1.98	2.14	2.21	2.34	2.26
WF	1.30	1.89	2.16	2.18	2.09	2.21
WF+CT	1.30	1.97	2.09	2.18	2.14	2.10

Table 4: Network Throughput (flits/cycle) — Hotspot Traffic

- All algorithms have substantially lower peak throughput under hotspot (~2.1-2.4 flits/cycle) compared to uniform traffic, confirming that the hotspot node creates a bottleneck that limits the entire network.
- WF+CT matches or slightly outperforms WF across most PIR values, with consistent throughput around 2.1-2.2 flits/cycle.

5.5 Saturation Summary

Algorithm	Uniform Sat. PIR	Uniform Sat. Latency (cycles)	Hotspot Sat. PIR	Hotspot Sat. Latency (cycles)
XY	0.04	838	0.02	689
WF	0.04	2137	0.02	715
WF+CT	0.04	1904	0.02	599

Table 5: Saturation Points Summary

6. Discussion

XY routing always takes the minimal-length path. After saturation, the network reaches a steady state where packets queue but still move efficiently along short deterministic routes. WF and WF+CT take longer paths in the adaptive phase (e.g., going North then East instead of directly East), increasing average hop count and thus consuming more link bandwidth per packet. This is a fundamental tradeoff: adaptivity costs path optimality.

Under hotspot traffic, buffers near node 0 fill up. Plain WF will route toward node 0's direction even when buffers are congested, because it has no visibility into congestion state. WF+CT's buffer-occupancy check detects when the route toward node 0 is congested and can select an alternate direction if available. At $\text{PIR}=0.02$ — the critical pre-saturation hotspot point — this filtering produces a 13% latency reduction, confirming the design goal.

Under low-to-medium uniform load, buffers are rarely more than 75% full. The congestion threshold condition triggers infrequently, so WF+CT defaults to the same behavior as WF. This is expected and desirable: the congestion logic adds overhead only when needed. The algorithms diverge meaningfully only when congestion actually exists — which is exactly the target scenario.

The West-First constraint (westward moves always handled deterministically by XY) is fully preserved in WF+CT. The safety fallback (returning all directions when all are congested) ensures the router never refuses all outputs, which would cause packet dropping. No deadlock was observed in any simulation run.

7. Conclusion

This project successfully implemented and evaluated three NoC routing algorithms for a 4x4 2D mesh topology using the Noxim simulator. The key contributions and findings are:

1. XY Routing establishes a strong baseline with the highest post-saturation throughput (4.43 flits/cycle uniform) due to minimal-length deterministic paths, but is most vulnerable to hotspot traffic — saturating at $\text{PIR}=0.02$ with 689 cycle latency.
2. West-First Routing provides adaptive capability within deadlock-free turn-model constraints. Under uniform traffic, it saturates with higher latency than XY at the same PIR (2137 vs 838 cycles at $\text{PIR}=0.04$), but under hotspot conditions performs comparably to XY at low loads.
3. West-First + Congestion Threshold, our novel extension, demonstrated measurable improvement over plain West-First specifically in hotspot conditions — 13% lower latency at the critical $\text{PIR}=0.02$ point (599 vs 715 cycles) and 11% improvement at uniform saturation (1904 vs 2137 cycles at $\text{PIR}=0.04$). The implementation preserves all West-First deadlock-freedom guarantees and adds negligible overhead.

The results confirm that lightweight, threshold-based congestion awareness can meaningfully improve adaptive routing performance in congestion-prone scenarios without requiring complex global state or virtual channels. Future work could explore dynamic threshold tuning and multi-hotspot scenarios.

Appendix A: WF+CT Source Code

File: src/routingAlgorithms/Routering_WEST_FIRST_CT.cpp

```
#include "Routing_WEST_FIRST.h"
#include "RoutingAlgorithm.h"
#include "RoutingAlgorithms.h"
#include "../Router.h"

class Routing_WEST_FIRST_CT : RoutingAlgorithm {
public:
    vector<int> route(Router * router, const RouteData & routeData);
    static Routing_WEST_FIRST_CT * getInstance();
private:
    Routing_WEST_FIRST_CT(){};
    static Routing_WEST_FIRST_CT * instance;
    static RoutingAlgorithmsRegister routingAlgorithmsRegister;
    static RoutingAlgorithm * xy;
};

RoutingAlgorithmsRegister
Routing_WEST_FIRST_CT::routingAlgorithmsRegister("WEST_FIRST_CT",
getInstance());

vector<int> Routing_WEST_FIRST_CT::route(Router * router, const RouteData &
routeData) {
    Coord current = id2Coord(routeData.current_id);
    Coord destination = id2Coord(routeData.dst_id);
    vector<int> directions;

    // West-First: westward moves handled by XY
    if (destination.x <= current.x || destination.y == current.y) {
        if(!xy) { xy = RoutingAlgorithms::get("XY"); }
        return xy->route(router, routeData);
    }
}
```

```

}
if (destination.y < current.y) {
    directions.push_back(DIRECTION_NORTH);
    directions.push_back(DIRECTION_EAST);
} else {
    directions.push_back(DIRECTION_SOUTH);
    directions.push_back(DIRECTION_EAST);
}
// Congestion filtering: skip directions >75% full
vector<int> uncongested;
for (int dir : directions) {
    unsigned int free = router->buffer[dir][0].getCurrentFreeSlots();
    unsigned int maxSize = router->buffer[dir][0].GetMaxBufferSize();
    if (free > maxSize / 4) uncongested.push_back(dir);
}
if (uncongested.empty()) return directions; // fallback
return uncongested;
}

```